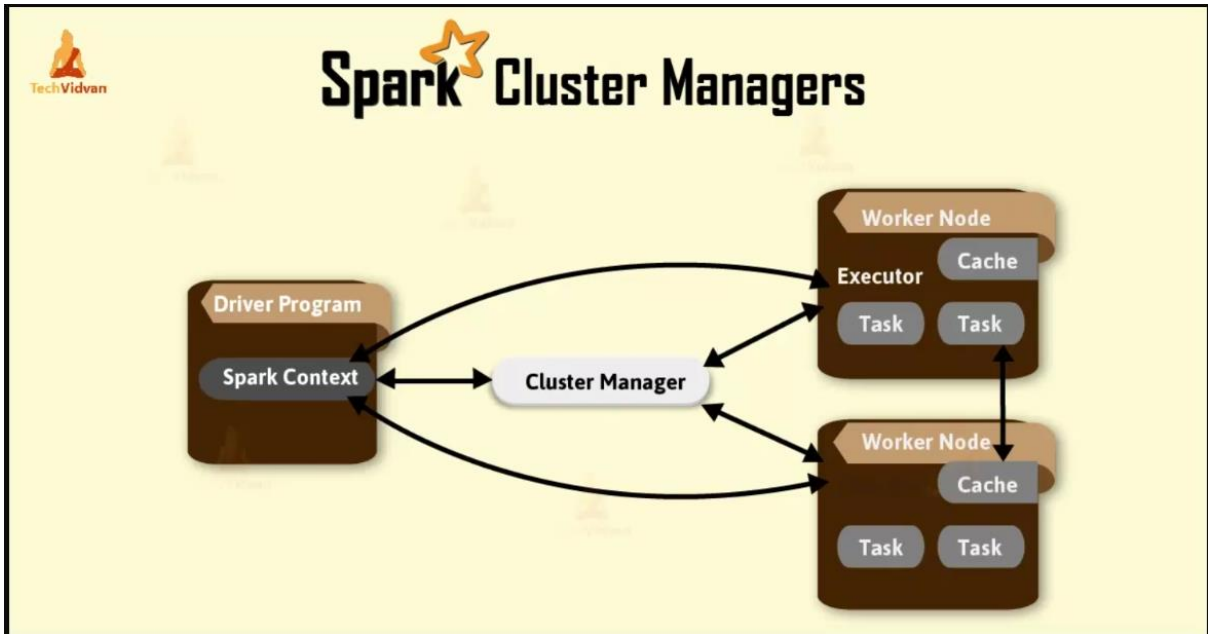


1. Spark Context (The Application Logic Orchestrator)

The SparkContext acts as the heart of your driver program. It represents the connection to a Spark cluster and is responsible for managing the internal workflow of your specific job.

- **Task Scheduling:** It breaks down jobs into stages and tasks, which it then dispatches to executors.
- **Resource Negotiation:** It communicates with the Cluster Manager to request the necessary resources to run these tasks.
- **Data Abstraction:** It is the primary gateway for creating Resilient Distributed Datasets (RDDs), accumulators, and broadcast variables.
- **Monitoring:** It provides a Web UI (typically on port 4040) to monitor the progress of your application's jobs and stages.

2. Cluster Manager (The Resource Provider)

The Cluster Manager is an external service that manages the pool of resources (nodes) available to all Spark applications running on the cluster.

- **Resource Allocation:** It decides which application gets how many cores and how much memory based on available capacity and priorities.
- **Executor Management:** Once an application is submitted, the Cluster Manager starts the executor processes on the worker nodes.
- **Pluggability:** Spark is "agnostic" to the cluster manager, meaning you can swap them out depending on your infrastructure needs.

SparkContext:

The driver program will create a **SparkContext** (SC) on a separate JVM. The role of SparkContext is:

1. SC connects your application to the Spark cluster. Without SparkContext, your app wouldn't be able to talk to the cluster to process data.
2. It asks the cluster manager for resources (like CPU and memory) to run your tasks.
3. Once it has the resources, SparkContext then divides your data into smaller chunks, known as RDDs (Resilient Distributed Datasets), and distributes these tasks across the cluster.
4. It ensures these tasks are distributed efficiently across the cluster, so your data is processed quickly and effectively.
5. It also keeps an eye on how your tasks are doing. It monitors the progress and provides diagnostic information if things don't go as planned.
6. If a task fails, SparkContext helps to figure out what went wrong and tries to fix the issue.

SparkSession

Spark 2.0 introduced a new entry point called **SparkSession** that essentially replaced both SQLContext and HiveContext. Additionally, it gave the developers immediate access to SparkContext.

1. One of the main goals of SparkSession was to provide a single entry point to all of Spark's functionality, simplifying the user experience. It merges the different SparkContext's into one, making it easier to manage and work with.

2. The SparkSession aimed to make Spark more accessible, especially for those working with structured data such as tables and SQL queries. It provides a more intuitive and user-friendly API.

How SparkContext & SparkSession are connected?

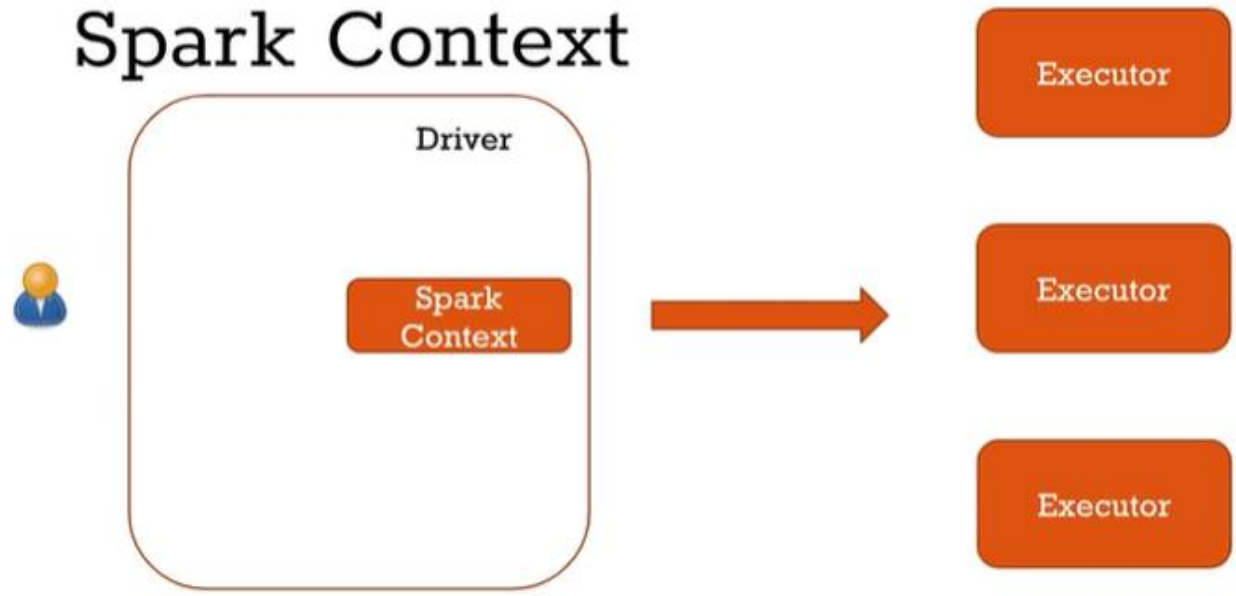
i. SparkSession is built on top of SparkContext, which means it encapsulates SparkContext.

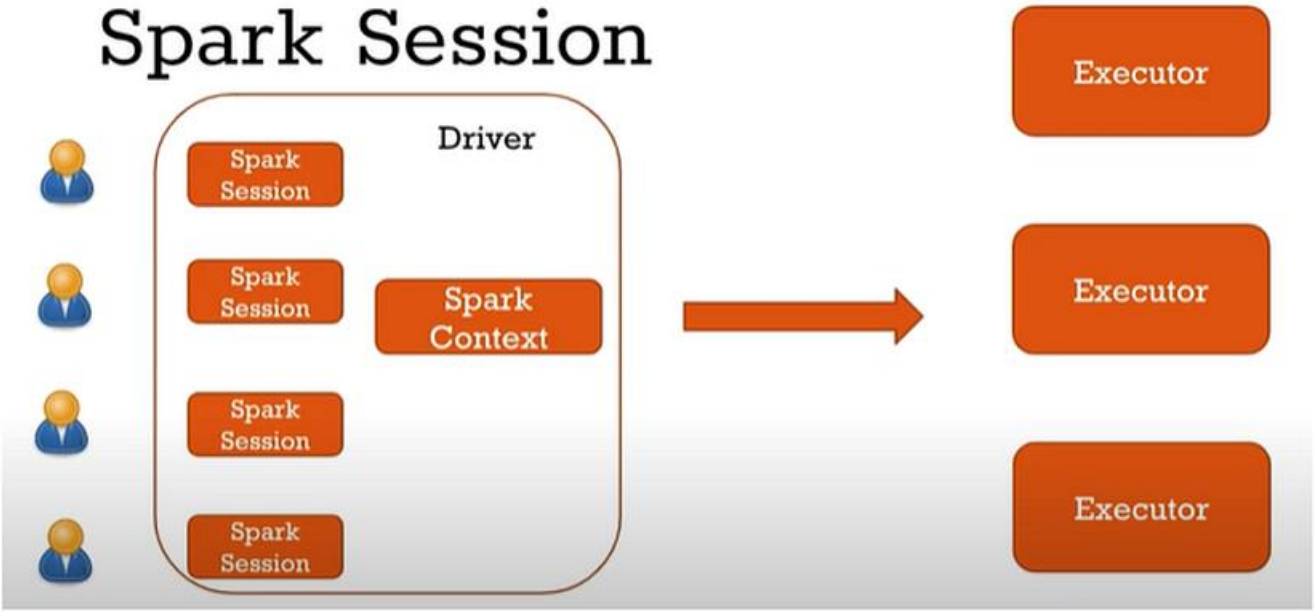
ii. SparkSession simplifies many tasks, SparkContext remains the fundamental entry point for RDD (Resilient Distributed Dataset) operations and some low-level API functionalities.

iii. In Spark 2.0 and later, when you create a SparkSession, it internally creates a SparkContext. Users typically don't need to explicitly create a SparkContext.

iv. For most high-level operations, especially dealing with structured data, you will be interacting with SparkSession. However, for RDD-based operations or lower-level functions, SparkContext is still relevant.

v. SparkSession maintains backward compatibility with older Spark applications that use SparkContext. This ensures that existing Spark applications continue to work seamlessly.

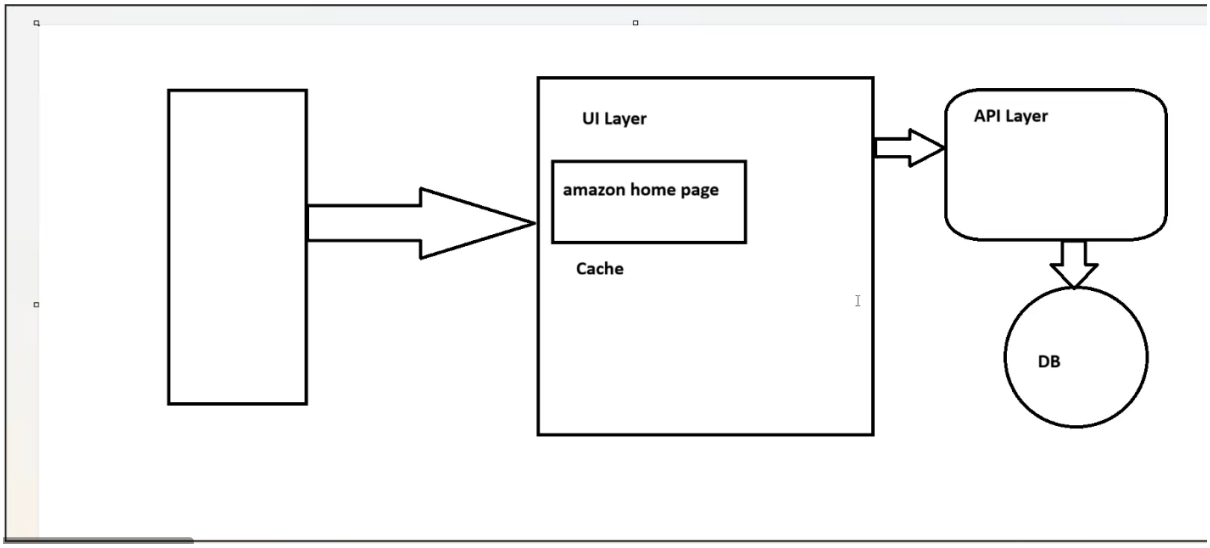




In earlier versions of spark (before 2.x) when multiple users want to use the cluster and run the queries on the same cluster they need to create multiple Spark Context's to fulfill the requirements of each user and they will be having isolated application on the cluster.

In later versions of spark, users can have multiple Spark Sessions but should have only one Spark Context. Every user can have his own Spark Session but there will be one SparkContext. (Spark Context represents 1 application), so there will be one application running here on the cluster which is going to run everything through executors.

Every user can have their own Spark Session, they can have their own properties ,their own configurations and they can have their own tables which are only visible for their own Spark Session.



WORKFLOW OF DATA PROCESSING

Redis Cache:

Integrating Redis with Apache Spark provides an ultra-low-latency caching layer that complements Spark's distributed processing power. While Spark has its own internal caching (df.cache()), using Redis as an external cache allows you to share data across different Spark applications, reduce re-computation costs for expensive datasets, and serve results to real-time APIs.

How Everything Works Together

Architecture flow:

```
User writes PySpark code
        ↓
Driver Node starts application
        ↓
SparkSession created
        ↓
SparkContext communicates with Cluster Manager
        ↓
Cluster Manager allocates Worker Nodes
        ↓
Executors process partitions of data
        ↓
Results returned to Driver
```

Example with a 1TB Dataset

Imagine processing a **1 TB dataset**.

Instead of one machine:

```
1 machine → slow
```

Spark divides it:

```
Worker 1 → 200 GB
Worker 2 → 200 GB
Worker 3 → 200 GB
Worker 4 → 200 GB
Worker 5 → 200 GB
```

Processing happens **in parallel**.

Client Mode

In **Client Mode**, the driver runs on the **client machine**.

Example:

```
Laptop / Notebook
   ↓
Driver runs here
   ↓
Cluster executes tasks
```

Typical use:

- Spark Shell
- PySpark notebooks
- Development

Example:

```
Google Colab
Jupyter Notebook
Databricks Notebook
```

Driver is outside the cluster.

Cluster Mode

In **Cluster Mode**, the driver runs **inside the cluster**.

Architecture:

```
Submit Job
   ↓
Cluster Manager starts Driver inside cluster
   ↓
Driver distributes tasks to workers
```

Benefits:

- better fault tolerance
- scalable production jobs
- no dependency on client machine

Used in:

- production pipelines
- scheduled Spark jobs
- enterprise clusters

Simple Diagram